# Plataforma de Inteligencia Artificial para la Detección Temprana de la Enfermedad de Alzheimer a través de la Voz (IAEAV)

Ángel Pérez Sempere

Miguel Ángel Teruel Martínez

Bárbara Escalante Belmonte

Javier García Sigüenza

Álvaro Navarro López-Menchero

Francisco Javier Sanchis Bernabeu

Cristian Javier Vera Arenas

# Índice general

**1. Descripción del producto** **1**

**2. Funcionalidades** **3**

**3. Código fuente** **5**

**4. Instalación paso a paso** **15**

**5. Demo** **18**

5.1. Frontend . . . . . 18

5.2. App . . . . . 23

*Página intencionalmente dejada en blanco.*

# 1
# Descripción del producto

La plataforma de Inteligencia Artificial (en adelante, IA) para la Detección Temprana de la Enfermedad de Alzheimer a través de la Voz (IAEAV) es una solución tecnológica orientada al cribado cognitivo remoto mediante el análisis automático de muestras de voz. Su desarrollo responde a la necesidad de disponer de herramientas escalables, accesibles y no invasivas que permitan apoyar la detección temprana del deterioro cognitivo, reduciendo la dependencia de evaluaciones exclusivamente presenciales y de recursos clínicos altamente especializados.

En cuanto a las partes de la plataforma, ésta integra una aplicación móvil guiada para la captura estructurada de tareas de habla, una infraestructura segura de gestión de datos. Además, inlcuye un sistema de procesamiento asíncrono para la inferencia mediante modelos de aprendizaje profundo. Este enfoque permite combinar la facilidad de uso para pacientes y profesionales con requisitos estrictos de seguridad, trazabilidad y reproducibilidad, aspectos especialmente relevantes en entornos biomédicos y clínicos.

En su conjunto, IAEAV se concibe como un ecosistema integral en el que la voz actúa como biomarcador digital no invasivo y de bajo coste. A partir de ella, la plataforma analiza información textual, acústica y multimodal para apoyar la identificación de patrones asociados a la enfermedad de Alzheimer. Entre sus principales aportaciones destacan el empleo de técnicas de IA Explicable (en adelante, XAI, por sus siglas en inglés *eXplainable Artificial Intelligence*), la validación de arquitecturas avanzadas para clasificación automática y la consolidación de un corpus clínico en español específicamente diseñado para este problema.

La plataforma ha sido diseñada para dar servicio tanto a equipos de investigación como a profesionales sanitarios, facilitando escenarios de telemonitorización y evaluación accesible. Del mismo modo, aporta beneficios directos a los pacientes, al permitir evaluaciones menos intrusivas, más cómodas y potencialmente más frecuentes, favoreciendo así el seguimiento longitudinal y la detección precoz.

Entre sus capacidades principales destacan:

- Captura guiada de tareas de voz mediante dispositivo móvil.
- Validación local de las grabaciones para mejorar la calidad de los datos.
- Gestión segura de audios y metadatos con mecanismos criptográficos.
- Procesamiento automático asíncrono para inferencia con IA.
- Análisis de modalidades de texto, audio y fusión multimodal.
- Incorporación de mecanismos de explicabilidad para interpretar la decisión de los modelos.
- Uso de corpus clínicos anonimizados para investigación y validación.

En conjunto, IAEAV constituye una plataforma robusta, innovadora y de alto valor aplicado, que contribuye a mejorar la detección temprana de la enfermedad de Alzheimer mediante tecnologías de voz e inteligencia artificial, reforzando al mismo tiempo la seguridad, la interpretabilidad y la utilidad clínica de los resultados.

# 2
# Funcionalidades

La plataforma IAEAV proporciona un conjunto de herramientas orientadas a la captura, protección, procesamiento y análisis de muestras de voz para el cribado temprano de la enfermedad del Alzheimer. Sus funcionalidades principales incluyen:

1. Captura guiada de datos de voz:
   - Registro de tareas de habla desde una aplicación móvil.
   - Guiado del usuario durante el proceso de grabación.
   - Validaciones acústicas locales para verificar la calidad de la muestra.
   - Envío tolerante a fallos mediante políticas de reintento ante redes inestables.
2. Gestión segura e interoperable de la información
   - Exposición de una API REST con separación entre endpoints y lógica de negocio.
   - Control de acceso basado en token.
   - Validación de metadatos asociados a cada grabación.
   - Registro exhaustivo de eventos para asegurar trazabilidad.
3. Protección criptográfica de audios y datos sensibles
   - Cifrado local del audio mediante clave simétrica AES-GCM.
   - Cifrado posterior de la clave simétrica con la clave pública del servidor mediante RSA-OAEP.
   - Recuperación segura de la clave únicamente por parte del servidor.
   - Garantía de confidencialidad e integridad tanto en tránsito como en almacenamiento.
4. Procesamiento asíncrono para inferencia con IA
   - Ejecución de tareas pesadas en segundo plano.
   - Control automático de calidad, por ejemplo mediante relación señal-ruido.
   - Activación desacoplada de los modelos de inferencia profunda.
   - Posibilidad de escalado horizontal dinámico sin bloquear la API principal.

5. Soporte de IA y explicabilidad.
   - La plataforma esta desarrollada para poder trabajar con diferentes modelos de IA.
   - Soporte para predecir el alzheimer a través del audio, texto y combinando ambas.
   - Integración de explicabilidad en las predicciones, pudiéndo aportar explicabilidad tanto en el uso de la información acústica como téxtual.
   - Permite analizar el impacto de los diferentes datos en la predicción.
   - Soporte para predecir si el paciente sufre Alzheimer.
   - Soporte para el cálculo del Mini-Mental State Examination (MMSE), indicándo las capacidad cógnitivas del paciente.

Estas funcionalidades permiten articular una plataforma completa para el cribado remoto de deterioro cognitivo, combinando facilidad de uso, garantías de seguridad y capacidad analítica avanzada, tanto para investigación como para aplicaciones clínicas.

# 3
# Código fuente

A continuación se describe la estructura de los repositorios de código fuente que componen la plataforma IAEAV.

Carpeta **iaeav_backend**:

El backend es una API REST construida sobre Flask 2.3 y Gunicorn, con PostgreSQL 15 como base de datos, Redis 7 como broker de tareas asíncronas (Celery) y cifrado de extremo a extremo basado en RSA-OAEP-256 y AES-256-GCM. El repositorio se organiza de la siguiente manera:

- **factory.py**: fábrica de la aplicación Flask (*Application Factory* pattern); inicializa las extensiones (SQLAlchemy, Flask-Migrate, CORS) y registra todos los Blueprints de la API.
- **models.py**: modelos SQLAlchemy que definen el esquema de la base de datos PostgreSQL: `User` (con rol, hash de contraseña y proveedor OAuth), `Recording` (grabaciones cifradas con su estado de análisis, clase predicha y transcripción) y `UploadSession` (sesiones de subida por fragmentos con control de integridad SHA-256).
- **celery_app.py**: configuración de Celery con Redis como broker; define la cola de tareas asíncronas para el procesamiento de audio.
- **wsgi.py**: punto de entrada WSGI para Gunicorn en producción (4 workers por defecto).
- **manage.py**: comandos CLI de Flask personalizados (`flaskcreate-admin`, integración con Flask-Migrate para migraciones de base de datos).
- **api/**: Blueprints REST organizados por dominio funcional, cada uno con su propia ruta base bajo `/api/v1/`:
  - **auth.py**: registro de usuario, login (devuelve JWT RS256), logout e invalidación de token, y flujo completo OAuth/OIDC con Google.
  - **users.py**: CRUD completo de usuarios (crear, listar, actualizar, eliminar), gestión de roles RBAC y consulta del perfil propio (`GET/PUT/users/me`).

  - **recordings.py**: subida cifrada por fragmentos en tres fases (init / chunk / complete) y consulta del estado y resultado del análisis de Alzheimer.
  - **uploads.py**: subida de archivos generales (ZIP, WAV, etc.), con listado, descarga y eliminación.
  - **crypto.py**: publicación del endpoint `GET/crypto/public-key` que devuelve la clave pública RSA para que el cliente cifre el audio antes de enviarlo.
  - **health.py**: endpoints de comprobación de estado para orquestadores Docker y balanceadores de carga (`/health/simple` y `/health` completo).
- **core/**: lógica transversal de la aplicación:
  - **config.py**: configuración por entorno (development, test, docker, production) leyendo variables de entorno; las claves criptográficas se cargan desde ficheros PEM.
  - **security.py**: generación y verificación de tokens JWT RS256 con PyJWT, decoradores de autorización RBAC (`@require_role`), tokens de subida de alcance limitado (caducan en 15 minutos) y verificación de integridad SHA-256 de los fragmentos recibidos.
  - **envelope.py**: cifrado de sobre (*Envelope Encryption*) — la clave de cifrado del dato (DEK) se genera aleatoriamente en cada subida, cifra el audio con AES-256-GCM, y la propia DEK se envuelve con RSA-OAEP-256 usando la clave pública del servidor; el servidor solo descifra temporalmente para el análisis.
  - **storage_local.py**: abstracción para la gestión del almacenamiento en disco (escritura atómica, lectura y borrado de archivos cifrados).
- **processing/**: módulos de procesamiento de señal de audio:
  - **snr.py**: cálculo de la relación señal-ruido (SNR) mediante detección de actividad vocal (VAD) y energía de la señal; las grabaciones que no superan el umbral mínimo son rechazadas antes del análisis IA.
  - **transcode.py**: transcodificación de audio al formato WAV con la frecuencia de muestreo y profundidad de bits requeridas por el modelo de inferencia.
  - **vad.py**: detección de actividad vocal para identificar y recortar segmentos de silencio al inicio y al final de la grabación.
- **tasks/analysis_task.py**: tarea Celery que se ejecuta de forma asíncrona tras completar la subida de una grabación con SNR válido; descifra el audio (RSA-OAEP + AES-GCM), lo envía al microservicio de inferencia IA y persiste el resultado (clase predicha, confianza, transcripción) en PostgreSQL.
- **deploy/**: archivos de despliegue Docker:
  - **docker-compose.prod.yml**: define los cuatro servicios del stack (PostgreSQL 15, Redis 7, API Flask y worker Celery).
  - **.env.example**: plantilla de todas las variables de entorno necesarias con sus descripciones.

- **init-scripts/**: scripts SQL ejecutados automáticamente al inicializar la base de datos PostgreSQL.
- **scripts/gen-keys.sh**: genera los cuatro ficheros PEM necesarios (`jwt_priv.pem`, `jwt_pub.pem`, `upload_priv.pem`, `upload_pub.pem`) en `deploy/keys/`.

Carpeta **iaeav_frontend**:

El frontend es una aplicación web de página única (*Single Page Application*) construida con React 19, TypeScript 5.9 en modo estricto y Vite 7 como *build tool*. Implementa autenticación JWT y un panel de control con contenido adaptado al rol del usuario. La estructura del repositorio es la siguiente:

- **App.tsx**: componente raíz que define el árbol de rutas con React Router 7; protege las rutas privadas redirigiendo al login si no existe sesión activa.
- **main.tsx**: punto de entrada de la aplicación; monta el `BrowserRouter` y el `AuthProvider` sobre el árbol de componentes.
- **contexts/AuthContext.tsx**: contexto global de autenticación; almacena el JWT en `localStorage`, lo decodifica para extraer el nombre de usuario, rol y email, expone las funciones de login y logout, y gestiona la expiración automática del token (1 hora).
- **services/apiService.ts**: capa de comunicación con la API REST del backend; centraliza todas las llamadas HTTP añadiendo automáticamente la cabecera `Authorization:Bearer<token>` y redirigiendo al login ante un error 401.
- **pages/**: vistas principales de la aplicación:
  - **LoginPage.tsx**: formulario de login con credenciales locales y botón de login con Google OAuth mediante redirección al backend.
  - **RegisterPage.tsx**: formulario de registro de nuevo usuario con validación en cliente.
  - **DashboardPage.tsx**: panel principal con pestañas cuyo contenido varía según el rol del usuario autenticado (ver tabla de funcionalidades por rol en el capítulo anterior).
  - **ProfilePage.tsx**: edición del perfil personal (nombre, email) y cambio de contraseña.
- **components/**: componentes reutilizables organizados por función:
  - **FileUploadForm.tsx**: formulario de subida de archivos con barra de progreso en tiempo real y validación del tipo de archivo en cliente.
  - **AdminUploadsTable.tsx**: tabla de archivos con permisos CRUD completos (admin y manager); permite ver todos los archivos, descargarlos y eliminarlos.
  - **OperatorUploadsTable.tsx**: tabla de archivos en modo solo lectura para el rol operador.

  - **UserUploadsTable.tsx**: tabla que muestra únicamente los archivos propios del usuario (participant y clinician).
  - **EncryptedRecordingsTable.tsx**: tabla de grabaciones cifradas con el estado del análisis IA (pendiente, procesando, completado) y acceso al resultado (clase predicha, confianza, transcripción); visible para manager y admin.
  - **UserManagement.tsx**: panel de gestión de usuarios con CRUD completo y estadísticas de uso; disponible únicamente para los roles manager y admin.
- **vite.config.ts**: configuración de Vite 7 que incluye el proxy de desarrollo para redirigir todas las peticiones `/api/*` al backend en `localhost:8000`, eliminando la necesidad de configurar CORS durante el desarrollo.
- **public/**: recursos estáticos servidos directamente: logo de la aplicación (`logo_iaeav-playstore.png`), logo de Google para el botón de OAuth y favicon.
- **dist/**: carpeta de salida generada por `npmrunbuild`, con los archivos HTML, CSS y JavaScript optimizados listos para servir con Nginx o cualquier servidor web.

Carpeta **iaeav_movil**:

Este módulo contiene el código fuente de una aplicación móvil desarrollada en Kotlin para plataforma Android, orientada a la captura, gestión y análisis de grabaciones de audio dentro del sistema IAEAV. La aplicación integra funcionalidades de autenticación de usuarios, grabación de audio, procesamiento seguro de la información y visualización de resultados, siguiendo una arquitectura modular basada en el patrón MVVM (Model-View-ViewModel).

El flujo general de la aplicación comienza con la autenticación del usuario mediante pantallas de login y registro. Una vez autenticado, el usuario puede acceder a funcionalidades principales como la grabación de audio, donde se capturan respuestas a distintas preguntas predefinidas. Estas grabaciones son procesadas, cifradas y almacenadas de forma segura, pudiendo posteriormente ser enviadas a un servidor remoto mediante servicios API. Finalmente, los resultados obtenidos son presentados en la interfaz, incluyendo funcionalidades adicionales como la generación de informes en formato PDF.

La estructura del código es la siguiente:

- **MainActivity.kt**: punto de entrada principal de la aplicación. Inicializa la interfaz basada en Jetpack Compose y configura la navegación entre pantallas mediante un grafo de navegación.
- **ui/navigation/NavGraph.kt**: define la estructura de navegación de la aplicación, estableciendo las rutas entre las distintas pantallas (login, registro, grabación, resultados, perfil, etc.).
- **ui/auth/**:
  - **LoginScreen.kt / RegisterScreen.kt**: implementan las interfaces de usuario para autenticación.

    - **LoginViewModel.kt / RegisterViewModel.kt**: gestionan la lógica de autenticación, comunicación con repositorios y control del estado de la UI.

- **ui/record/**:
    - **RecordScreen.kt**: interfaz para la grabación de audio por parte del usuario.
    - **RecordViewModel.kt**: gestiona la lógica de grabación, interacción con componentes de audio y coordinación con el repositorio.

- **ui/results/**:
    - **ResultScreen.kt**: muestra los resultados obtenidos tras el procesamiento de las grabaciones.
    - **ResultViewModel.kt**: maneja la lógica asociada a la obtención y presentación de resultados.
    - **GeneratePDF.kt**: funcionalidad para generar informes en formato PDF a partir de los resultados.
    - **ExpandableCard.kt**: componente UI reutilizable para mostrar información expandible.

- **ui/profile/**:
    - **ProfileScreen.kt / ProfileViewModel.kt**: permiten la visualización y gestión de la información del usuario.

- **ui/info/**:
    - **InfoScreen.kt**: pantalla informativa sobre la aplicación.

- **ui/loading/**:
    - **LoadingScreen.kt / LoadingViewModel.kt**: gestionan estados de carga durante operaciones asíncronas.

- **ui/theme/**: contiene la configuración visual de la aplicación (colores, tipografías y estilos) utilizada mediante Jetpack Compose.
- **data/api/**:
    - **AuthApi.kt, CryptoApi.kt, RecordingsApi.kt**: definen las interfaces para la comunicación con servicios remotos (autenticación, cifrado y gestión de grabaciones).

- **data/model/**:
    - **AuthDtos.kt, RecordingDtos.kt, RecordingDetailDto.kt**: contienen las clases de transferencia de datos (DTOs) utilizadas para mapear las respuestas de la API.

- **data/repo/**:
    - **AuthRepository.kt, RecordingRepository.kt**: implementan la lógica de acceso a datos, actuando como intermediarios entre la capa de red y los ViewModels.

- **core/**:
  - **ServiceLocator.kt**: implementa un patrón de inyección de dependencias manual para proporcionar instancias compartidas.
  - **audio/AudioRecorder.kt**: gestiona la captura de audio desde el dispositivo.
  - **crypto/**:
    - **AesGcm.kt**: implementación de cifrado simétrico mediante AES-GCM.
    - **Rsa.kt / RsaUtils.kt**: utilidades de cifrado asimétrico mediante RSA.
    - **Base64u.kt**: funciones auxiliares de codificación Base64.
  - **storage/SecurePrefs.kt**: gestiona el almacenamiento seguro de datos sensibles en el dispositivo.
- **workers/UploadWorker.kt**: implementa tareas en segundo plano utilizando WorkManager para la subida de grabaciones al servidor.
- **res/**: contiene los recursos de la aplicación (imágenes, audios, estilos, strings, etc.).
- **AndroidManifest.xml**: define la configuración global de la aplicación, incluyendo permisos y componentes.
- **build.gradle.kts**: archivos de configuración de compilación y dependencias del proyecto.

Carpeta **ia_multimodal**:

Este módulo contiene el código para el entrenamiento y evaluación de un modelo multimodal orientado a la detección de Alzheimer y a la estimación de deterioro cognitivo a partir de información lingüística y acústica derivada de la voz. A diferencia de un sistema puramente basado en audio o en texto, este entorno combina transcripciones automáticas con embeddings acústicos extraídos mediante modelos ASR, permitiendo estudiar de forma conjunta el contenido verbal y las características temporales del audio.

El flujo general del codigo parte de archivos de audio, que son procesados mediante reconocimiento automático del habla para generar, por un lado, transcripciones textuales y, por otro, representaciones acústicas. A partir de estas dos fuentes, el sistema entrena el modelo y lo evalua.

La estructura del codigo es la siguiente:

- **classify_train.py**: script principal de entrenamiento para la tarea de clasificación. Implementa un modelo multimodal que combina embeddings textuales obtenidos con BERT y embeddings acústicos, realizando entrenamiento y evaluación mediante validación cruzada estratificada. Incluye cálculo de métricas, selección del mejor modelo por fold y almacenamiento de resultados y configuraciones de entrenamiento.

- **mmse_train.py**: script de entrenamiento para la tarea de regresión de la puntuación MMSE. Utiliza la arquitectura multimodal para predecir una variable continua asociada al estado cognitivo, optimizando con pérdida MSE y evaluando mediante RMSE.
- **audio_asr_to_text.py**: utilidades de preprocesamiento que transforman audios en transcripciones automáticas y embeddings acústicos. Soporta distintos flujos de trabajo, que posteriormente son utilizados por el modelo.
- **recognize.py**: implementación de los componentes de reconocimiento automático del habla. Incluye la lógica necesaria para cargar modelos ASR, decodificar audio y obtener tanto la transcripción como las representaciones utilizadas como entrada acústica del sistema multimodal.
- **check_pause_length.py**: script de análisis exploratorio sobre pausas y patrones de segmentación en las transcripciones. Permite estudiar la distribución de silencios o separadores, así como analizar umbrales y propiedades estadísticas potencialmente relevantes para la caracterización del habla.
- **dataloaders.py**: contiene las implementaciones de los conjuntos de datos utilizados en las pruebas.
- **layers.py**: define diferentes capas empleadas en el modelo multimodal.
- **core.py**: funciones auxiliares de procesamiento textual. Implementa la carga y normalización de transcripciones generadas por ASR, así como reglas para convertir secuencias de pausas en signos de puntuación o separadores léxicos, facilitando la obtención de versiones limpias del texto.
- **helpers.py**: utilidades generales del proyecto, principalmente para la gestión del dispositivo de ejecución (CPU/GPU).
- **pre_train_weights/**: recursos asociados a los modelos ASR preentrenados:
  - **dict.ltr.txt**: diccionario de símbolos utilizado durante la decodificación de modelos de reconocimiento.
  - **download_weights.sh**: script para descargar pesos preentrenados de modelos de ASR.
- **requirements.txt**: listado de dependencias necesarias para reproducir el entorno experimental, incluyendo bibliotecas de deep learning, procesamiento de audio, transformers y utilidades de evaluación.

Carpeta **iaeav_ia_inference**:

El microservicio de inferencia IA es una API REST construida con Flask 2.3 que implementa el pipeline completo de detección de indicadores de Alzheimer a partir de grabaciones de voz. Combina reconocimiento automático del habla (ASR) con el clasificador multimodal ABERTA, con soporte para inglés (fairseq) y español (HuggingFace CTC). La estructura del repositorio es la siguiente:

- **src/api/app.py**: API principal con dos endpoints: `GET/health` (comprueba que el modelo está cargado y listo para recibir peticiones) y `POST/predict` (ejecuta el

pipeline completo de inferencia sobre un archivo de audio en formato WAV, MP3 o FLAC).

- **src/inference/inference_handler.py**: handler unificado que orquesta el pipeline ASR + WERT; carga los pesos del modelo al inicio, ejecuta el reconocimiento automático del habla con Wav2Vec2, extrae los embeddings del audio, los concatena con los embeddings BERT del texto transcrito, y devuelve la clase predicha (0 = sin indicadores, 1 = indicadores de Alzheimer), la confianza y la transcripción.
- **src/config/settings.py**: configuración centralizada mediante variables de entorno (`WEIGHTS_DIR`, `ASR_BACKEND`, `BERT_MODEL_NAME`, `WERT_FILENAME`, etc.) con valores por defecto que permiten el arranque sin configuración adicional en desarrollo local.
- **src/diarization.py**: pipeline de diarización basado en WhisperX y pyannote.audio; separa automáticamente los turnos de habla del clínico y del paciente en conversaciones completas, de modo que solo el audio del paciente se procesa por el modelo de clasificación.
- **src/diarization_api.py**: servicio Flask independiente que expone la funcionalidad de diarización como microservicio; el servicio de inferencia principal lo consulta cuando recibe un audio con la cabecera `X-Audio-Type:full`.
- **Dockerfile** y **Dockerfile.diarization**: imágenes Docker para la API de inferencia y para el servicio de diarización, respectivamente (ambas sobre Python 3.8 con CUDA opcional).
- **docker-compose.dual.yml**: orquestación del despliegue dual: `api_en` (puerto 2020, backend ASR fairseq, inglés) y `api\_es` (puerto 2022, backend ASR HuggingFace CTC, español); el backend de la plataforma selecciona el servicio según el idioma de la prueba.
- **download-weights.sh**: script que descarga automáticamente los aproximadamente 12 GB de pesos del modelo: el clasificador WERT *fine-tuned* desde el NAS del proyecto, Wav2Vec2 inglés desde fairseq/Meta AI, y BETO y Wav2Vec2 español *fine-tuned* desde HuggingFace.
- **requirements.txt** y **requirements-diarization.txt**: dependencias Python del servicio de inferencia y del servicio de diarización, respectivamente.
- **training/wav2bert-feature-rework/**: código completo de entrenamiento del modelo WERT para inglés usando el framework *fairseq*; incluye dataloaders, definición de capas personalizadas (**layers.py**), cabeza clasificadora (**model_head.py**), bucle de entrenamiento (**full_train.py**), cálculo de métricas (**cal_metrics.py**) y script de descarga de pesos preentrenados.
- **training/wav2bert-feature-spanish/**: variante española del código de entrenamiento, adaptada para usar Wav2Vec2-Large-XLSR-53 y BETO desde HuggingFace; añade **generate_dataset.py** para la generación del conjunto de datos en español y **download_model.py** para la descarga automática de los modelos base.

Carpeta **ia_texto**:

- **main.py**: archivo que permite ejecutar el cuerpo principal del código con 4 modelos basados en la arquitectura *Transformer* incluyendo el entrenamiento y la ejecución de éstos.
- **utils.py**: archivo que incorpora funciones de distintas utilidades para facilitar y optimizar el funcioanmiento del código.
- **model.py**: archivo que incluye funciones para el entrenamiento y la predicción de los modelos.
- **paths.py**: archivo que permite gestionar las rutas relativas para la gestión del código y los directorios que éste emplea.
- **data.py**: archivo enfocado en la carga de los datos.
- **data_src**: los datos de entrada del dataset empleado.

Carpeta **ia_audio**:

Este módulo contiene el código de investigación para el desarrollo y evaluación de nuevos modelos de inteligencia artificial orientados a la detección de indicadores de Alzheimer a partir de señales de voz. A diferencia del sistema en producción, este entorno permite experimentar con distintas configuraciones de modelos, representaciones espectrales y estrategias de entrenamiento de forma reproducible.

El flujo completo implementado parte de señales de audio en formato WAV, que son procesadas dinámicamente durante el entrenamiento. En lugar de utilizar espectrogramas precomputados, el modelo trabaja directamente con el audio completo, generando ventanas temporales y sus correspondientes representaciones espectrales dentro de la propia red neuronal, lo que permite una mayor flexibilidad y eficiencia experimental.

La estructura del repositorio es la siguiente:

- **train.py**: script principal de entrenamiento que implementa todo el pipeline experimental: carga de datos, construcción del modelo, entrenamiento con soporte para *mixed precision*, validación, test y evaluación mediante múltiples métricas (accuracy, recall, F1, AUC, entre otras). Además, gestiona la ejecución mediante validación cruzada estratificada y el almacenamiento de resultados, históricos y predicciones.
- **datasets/**: contiene las implementaciones de los conjuntos de datos utilizados:
  - **iaeav_dataset.py**: define el dataset principal, que carga archivos WAV completos, realiza re-muestreo, conversión a mono y construye particiones estratificadas en train/val/test mediante *Stratified K-Fold*.
  - **iaeav_dataset_balanced.py**: variante que implementa balanceo de clases mediante *undersampling* en el conjunto de entrenamiento, manteniendo la distribución original en validación y test.
- **models/**: implementaciones de los modelos de deep learning:
  - **resnext_audio.py**: adaptación de arquitecturas ResNeXt para el procesamiento de espectrogramas, incluyendo mecanismos de preservación temporal y atención en el eje temporal.

- **wav_windowing_stochastic_model.py**: modelo principal de investigación que recibe audio completo y genera dinámicamente ventanas aleatorias durante el entrenamiento, calculando sus espectrogramas y agregando las predicciones durante la inferencia. Este enfoque permite reducir el coste computacional manteniendo la representatividad de la señal.
- **wav_windowing_model.py**: variante determinista que procesa todas las ventanas del audio y agrega sus predicciones mediante estrategias como media o máximo.

- **utils/**:
  - **logger.py**: implementación de un sistema de logging ligero que permite registrar la evolución del entrenamiento, métricas y eventos relevantes tanto en consola como en fichero, facilitando la trazabilidad de los experimentos.

# 4
# Instalación paso a paso

A continuación se describe el proceso completo para desplegar la plataforma IAEAV en un entorno local o de producción, partiendo desde cero.

1. Instalar Docker.

- Descargar e instalar Docker Desktop desde `https://docs.docker.com/get-docker/` (disponible para Windows, macOS y Linux). En servidores Linux sin interfaz gráfica se puede instalar únicamente Docker Engine y el plugin Docker Compose V2.
- Verificar que la instalación es correcta ejecutando `docker-{}-version` y `docker composeversion` en un terminal. Se requiere Docker Compose V2 (el comando es `dockercompose`, sin guion).

2. Configurar variables de entorno.

- Copiar la plantilla de variables del backend con `cpbackend/deploy/.env.examplebackend/deploy/.env` y editar el fichero resultante con las credenciales reales: contraseña de PostgreSQL (`POSTGRES\_PASSWORD`), contraseña de Redis (`REDIS\_PASSWORD`), clave secreta de Flask (`SECRET\_KEY`) y contraseña del administrador inicial (`ADMIN\_PASSWORD`).
- Generar las cuatro claves criptográficas ejecutando `bashbackend/deploy/scripts/gen-keys.sh` desde la raíz del repositorio del backend. El script crea los ficheros `jwt\_priv.pem`, `jwt\_pub.pem`, `upload\_priv.pem` y `upload\_pub.pem` en `backend/deploy/keys/`; estas claves son las que usa el backend para firmar los JWT y para el cifrado de sobre de los audios.

3. Arrancar servicio de IA.

- Descargar los pesos del modelo (aproximadamente 12 GB en total) ejecutando `cd ai-inference/\&\&./download-weights.sh`. El script obtiene automáticamente, en este orden: el clasificador WERT *fine-tuned* desde la carpeta del proyecto, el modelo Wav2Vec2 para inglés desde el repositorio fairseq de Meta AI, el modelo

BETO (BERT en español) desde HuggingFace (`dccuchile/bert-base-spanish-wwm-uncased`) y el Wav2Vec2 español *fine-tuned* desde HuggingFace (`jonatasgrosman/wav2vec2-large-xlsr-53-spanish`).

- Levantar los contenedores del servicio de inferencia con `dockercompose-fdocker-compose.dual.ymlup-d-{}-build`. Esto arranca dos instancias del servicio: `api\_en` en el puerto 2020 (inglés, backend ASR fairseq) y `api\_es` en el puerto 2022 (español, backend ASR HuggingFace CTC). Verificar que ambos están listos con `curlhttp://localhost:2020/health` y `curlhttp://localhost:2022/health`; la respuesta esperada es `\{``status'':``ready''\}`.

4. Arrancar backend.

- Construir y levantar los cuatro servicios del backend (PostgreSQL 15, Redis 7, API Flask y worker Celery) con `cdbackend/\&\&dockercompose-fdeploy/docker-compose.prod.ymlup-d-{}-build`. En el primer arranque, Docker descarga las imágenes base y construye el contenedor del backend; los contenidos de la base de datos se persisten en un volumen Docker.
- Verificar que el backend responde correctamente ejecutando `curlhttp://localhost:8000/api/v1/health/simple`; la respuesta esperada es `\{"status":"ok"\}`. Para un diagnóstico más completo (estado de la base de datos, almacenamiento y versión) usar `curlhttp://localhost:8000/api/v1/health`. Si es el primer despliegue, crear el usuario administrador con `dockercompose-fdeploy/docker-compose.prod.ymlexecbackendflaskcreate-admin`.

5. Arrancar frontend.

- Instalar las dependencias del proyecto (requiere Node.js 18 o superior y npm 9 o superior): `cdfrontend/\&\&npminstall`.
- Para desarrollo local, arrancar el servidor de desarrollo con `npmrundev`; la aplicación queda disponible en `http://localhost:5173` y las peticiones a `/api/*` se redirigen automáticamente al backend en `localhost:8000` gracias al proxy configurado en `vite.config.ts`. Para producción, compilar con `npmrunbuild` y servir la carpeta `dist/` con Nginx (o cualquier servidor web estático), configurando el proxy `/api/*` hacia el backend y el *fallback* de rutas a `index.html` para que React Router funcione correctamente.

6. Instalar APK.

- Obtener el fichero APK de la aplicación Android desde el repositorio de la aplicación móvil o desde el NAS del proyecto.
- Instalar el APK en un dispositivo Android 8.0 (API 26) o superior habilitando previamente la opción «Instalar aplicaciones de fuentes desconocidas» en los ajustes de seguridad del dispositivo (o, en Android 8+, concediendo el permiso directamente al explorador de archivos o al gestor desde el que se instale). Al abrir la aplicación por primera vez, introducir la dirección del servidor backend en la pantalla de configuración.

7. Gestión desde frontend.

- Acceder al panel de administración en `http://localhost:5173` en modo desarrollo, o en el dominio configurado en Nginx en producción. Iniciar sesión con las credenciales del usuario administrador creado en el paso 4, o mediante Google OAuth si está configurado.
- Desde el panel de administración es posible crear nuevos usuarios asignando el rol correspondiente (`participant`, `clinician`, `operator`, `manager` o `admin`), gestionar los archivos subidos, consultar las grabaciones cifradas con su resultado de análisis IA, y descargar los informes. Para tareas de mantenimiento adicionales (reinicio de servicios, migraciones de base de datos, consulta de logs) referirse a los comandos descritos en el README del backend.

# 5

# Demo

## 5.1. Frontend

A continuación se muestra un recorrido por las principales pantallas del panel web de la plataforma IAEAV.

### Inicio de sesión

La primera pantalla que encuentra el usuario es la página de inicio de sesión (Figura 5.1). En la parte superior se muestra el logotipo de la plataforma. El formulario dispone de un único campo de identificación que acepta tanto el nombre de usuario como el correo electrónico, seguido del campo de contraseña y el botón *Entrar*. Bajo un separador «O», se ofrece el botón *Iniciar sesión con Google*, que redirige al flujo OAuth del backend y devuelve un JWT propio de la plataforma al completarse. Un enlace al pie conduce al formulario de registro para usuarios nuevos.

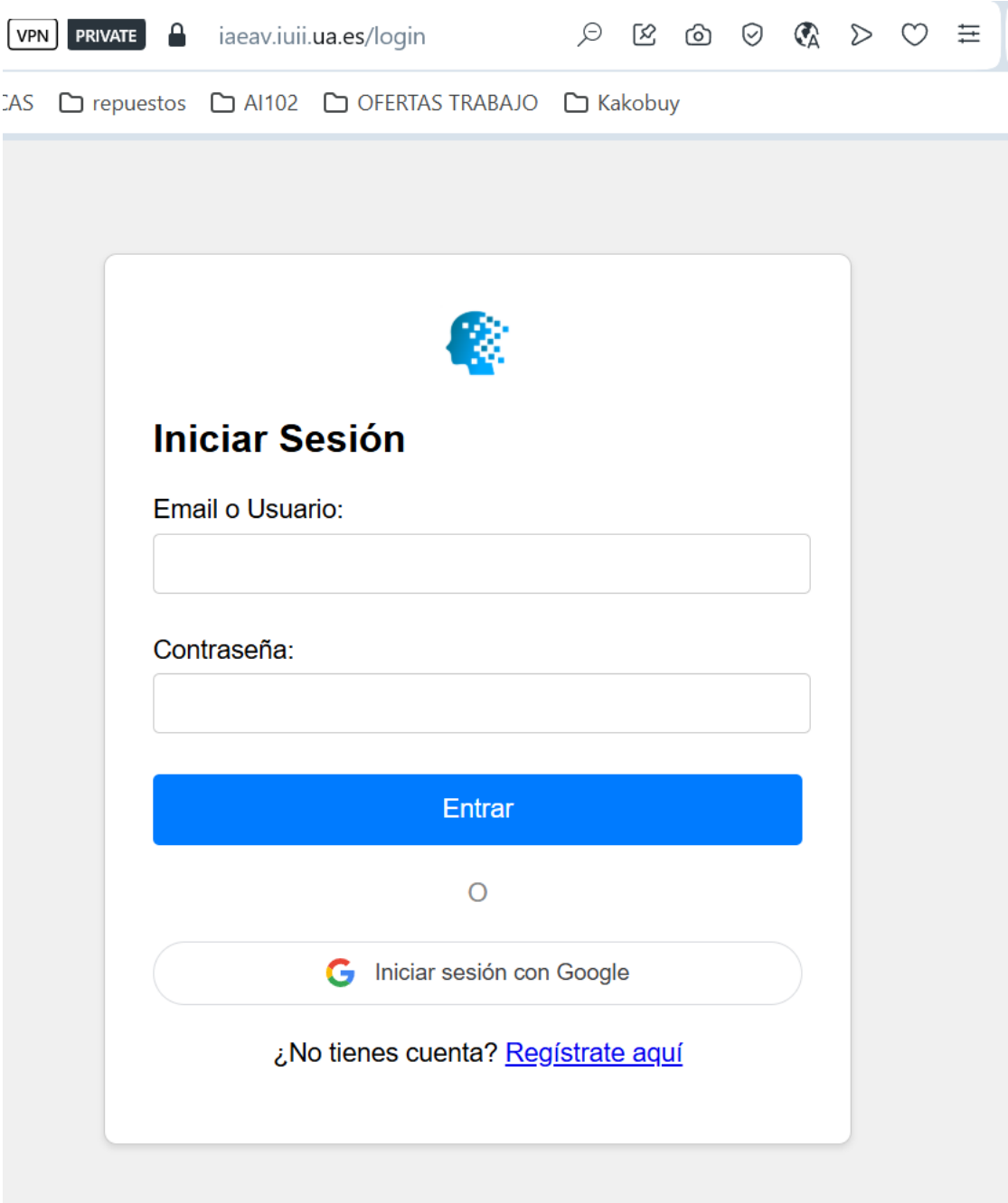


**Figura 5.1:** *Pantalla de inicio de sesión del panel web.*

### Panel de control

Una vez autenticado, el sistema decodifica el JWT, extrae el rol del usuario y lo redirige al panel de control (Figura 5.2). La cabecera muestra el título «Panel de Control» a la izquierda y, a la derecha, el nombre del usuario acompañado de un *badge* de color que indica su rol: rojo para `admin`, naranja para `manager`, morado para `operator`, verde para `clinician` y gris para `participant`. Al hacer clic sobre el área del usuario se despliega un menú con las opciones *Mi Cuenta* y *Cerrar Sesión*.

Inmediatamente bajo la cabecera se encuentra la barra de pestañas de navegación.

Las pestañas disponibles varían según el rol:

- **Subir Archivos**: visible para todos los roles; contiene el formulario de subida.
- **Mis Archivos / Gestión de Archivos**: muestra «Mis Archivos» para `participant` y `clinician`, y «Gestión de Archivos» para `operator`, `manager` y `admin`.
- **Grabaciones Cifradas**: visible únicamente para `manager` y `admin`.
- **Gestión de Usuarios**: visible únicamente para `manager` y `admin`.

La pestaña activa se resalta en azul con un subrayado inferior.

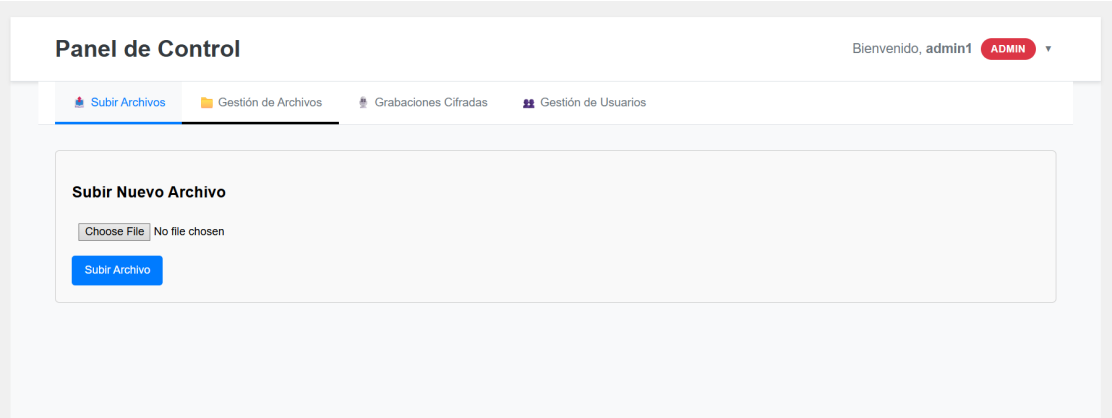


**Figura 5.2:** *Panel de control con la pestaña «Subir Archivos» activa. En la cabecera se aprecia el badge de rol del usuario.*

## Subida de archivos

La pestaña *Subir Archivos* muestra el componente `FileUploadForm` (Figura 5.3). El usuario selecciona un fichero de entre los formatos admitidos (ZIP, WAV, MP3, OGG, M4A); al seleccionarlo se muestra el nombre y el tamaño en megabytes. Al pulsar *Subir Archivo*, el botón cambia su texto a «Subiendo... X %» y aparece una barra de progreso nativa del navegador que refleja el avance de la transferencia en tiempo real. Al finalizar, se muestra un mensaje de éxito o de error según el resultado devuelto por el backend.

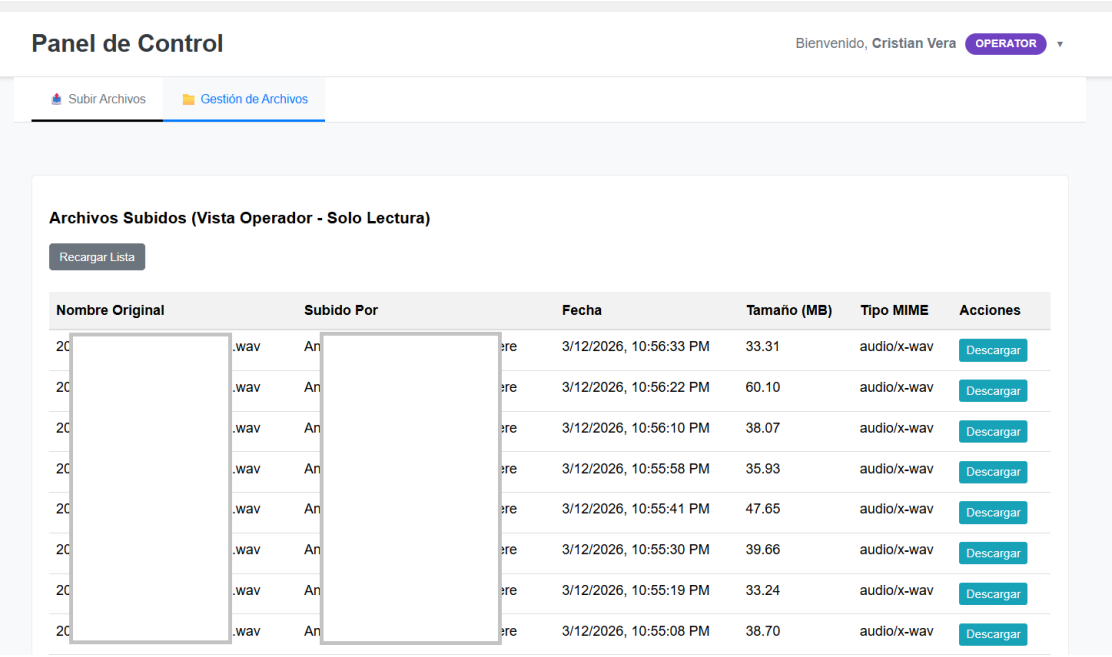


**Figura 5.3:** *Vista de subida de archivos Subidos.*

## Grabaciones cifradas

La pestaña *Grabaciones Cifradas*, accesible para `manager` y `admin`, presenta una tabla paginada con todas las grabaciones recibidas por la plataforma (Figura 5.4). Las columnas muestran el identificador de la grabación (truncado), el pseudónimo del participante, el identificador de tarea, la fecha y hora de recepción, la relación señal-ruido en decibelios y la duración en segundos. La columna de acciones ofrece el botón *Descargar* (*Descifrar*), que solicita al backend el audio descifrado. La paginación permite navegar entre páginas cuando el volumen de grabaciones lo requiere.

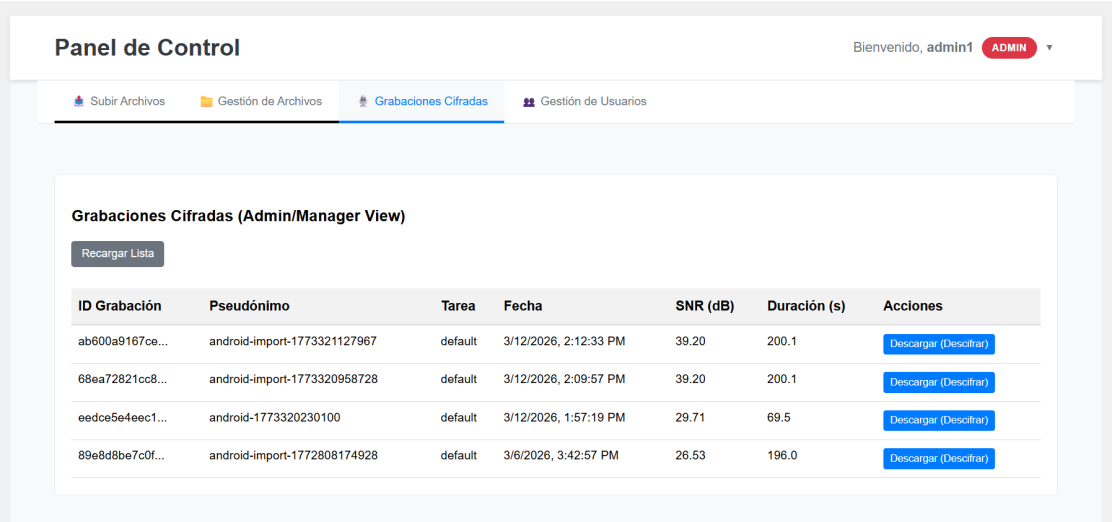


**Figura 5.4:** *Tabla de grabaciones cifradas con metadatos de calidad de audio.*

### Gestión de usuarios

La pestaña *Gestión de Usuarios* está disponible para `manager` y `admin` (Figura 5.5). En la parte superior se muestra un panel de estadísticas con una cuadrícula de tarjetas: la primera indica el total de usuarios registrados y las siguientes desglosan el recuento por cada rol, con el color identificativo de cada uno. Bajo las estadísticas aparecen los botones *Crear Nuevo Usuario* y *Actualizar Lista*. Al pulsar el primero se despliega un formulario inline con campos para introducir el nombre de usuario, la contraseña y un selector de rol. La tabla de usuarios incluye las columnas ID, Usuario, Email, Rol (badge coloreado), fecha de creación y las acciones *Editar* (verde) y *Eliminar* (rojo). Al pulsar *Editar* se abre un modal centrado que permite cambiar el rol del usuario y, opcionalmente, establecer una nueva contraseña.

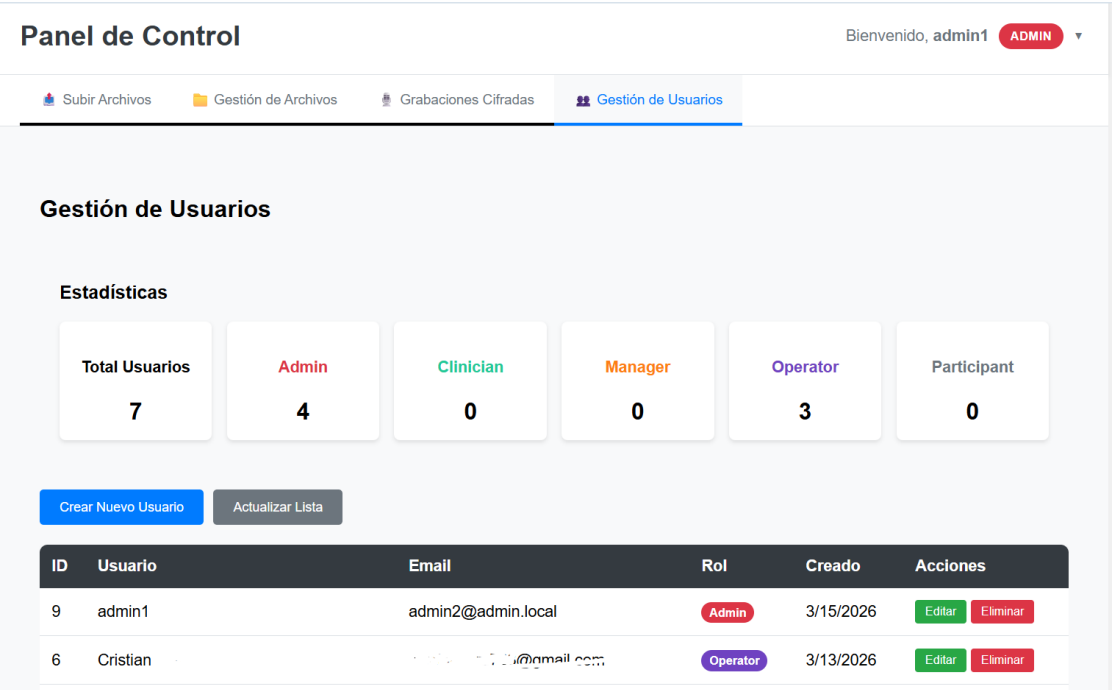


**Figura 5.5:** *Panel de gestión de usuarios con estadísticas por rol y tabla con acciones de edición.*

### Perfil de usuario

Desde el menú desplegable de la cabecera, la opción *Mi Cuenta* lleva a la página de perfil (Figura 5.6). El formulario superior permite editar el nombre de usuario y el correo electrónico; para cuentas autenticadas mediante Google, el campo de email aparece deshabilitado con una nota explicativa. El botón *Guardar Cambios de Perfil* actualiza los datos y refresca el token JWT para que el nombre de usuario se refleje de inmediato en la cabecera. Para cuentas locales (no Google) se muestra una sección adicional *Cambiar Contraseña* con los campos contraseña actual, nueva contraseña y confirmación. Un enlace al pie permite volver al panel de control.

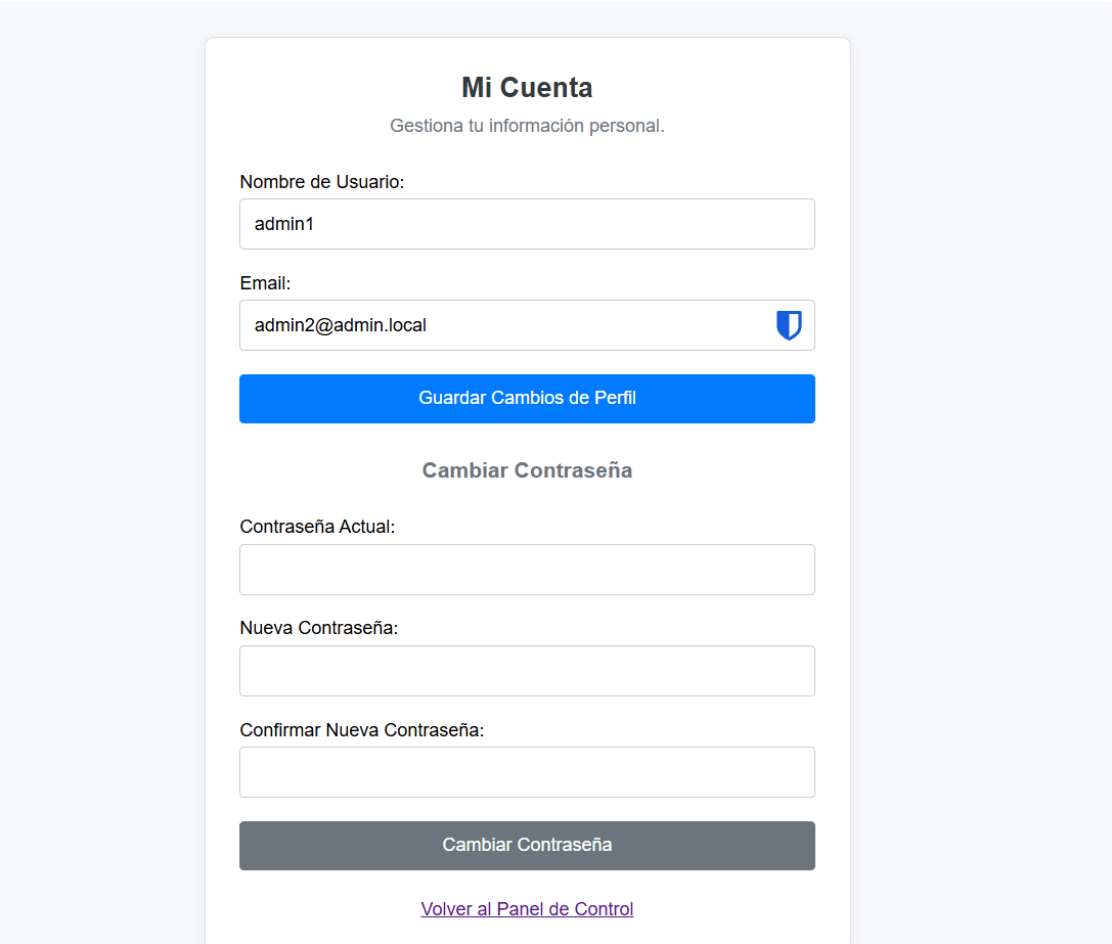


**Figura 5.6:** *Página de perfil con formularios de edición de datos y cambio de contraseña.*

## 5.2. App

A continuación, se muestra un recorrido por las principales pantallas de la aplicación móvil desarrollada.

En primer lugar, la Figura 5.7 muestra la pantalla de inicio de sesión de la aplicación, donde el usuario introduce sus credenciales para acceder al sistema. Una vez autenticado correctamente, el usuario accede a la pantalla principal de la aplicación (Figura 5.8), desde la cual puede iniciar la prueba, así como cerrar sesión y ver su cuenta.

Durante la realización de la prueba, la aplicación presenta diferentes preguntas que el usuario debe responder mediante audio. En la Figura 5.9 se muestra un ejemplo de una de las preguntas que aparecen durante la evaluación.

Una vez finalizada la prueba, la aplicación muestra una pantalla de carga (Figura 5.10) mientras el modelo de inteligencia artificial procesa el audio grabado y realiza el análisis correspondiente.

Finalmente, la aplicación presenta la pantalla de resultados (Figura 5.11), donde el usuario puede consultar la predicción realizada por el modelo, junto con la transcripción del audio de la prueba y una explicación del resultado proporcionado por el

sistema, así como descargar dicho resultado en PDF si asi lo desea.

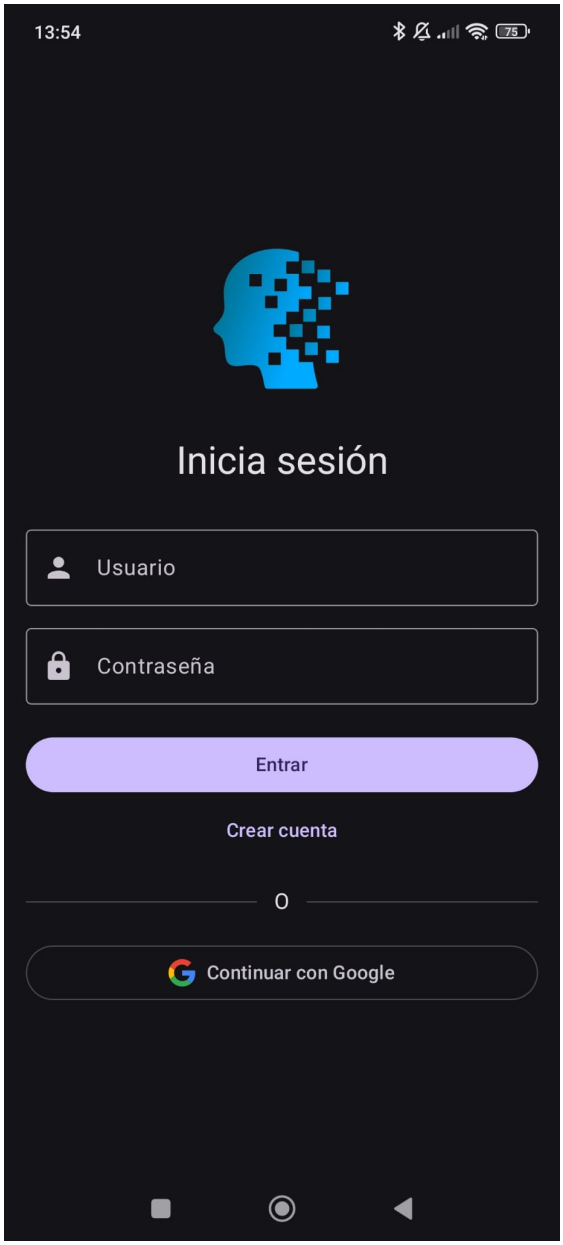


**Figura 5.7:** *Captura de la pantalla de inicio de sesión de la aplicación.*

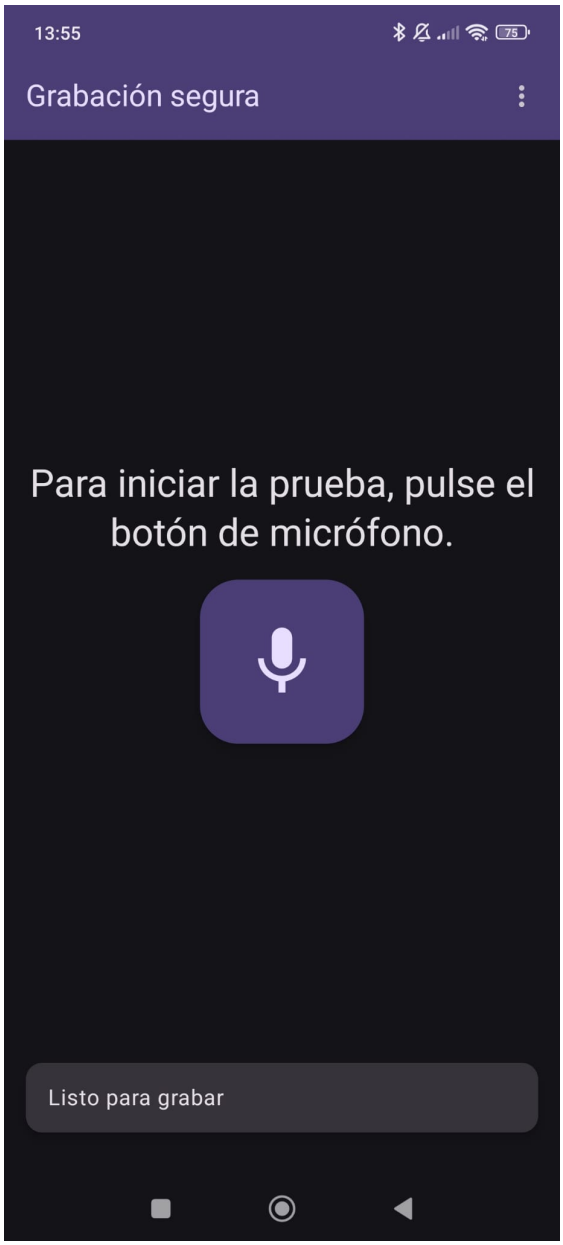


**Figura 5.8:** *Pantalla inicial de la aplicación a la que accedes al iniciar sesión.*

**Figura 5.9:** *Pantalla que muestra una de las preguntas de la prueba.*

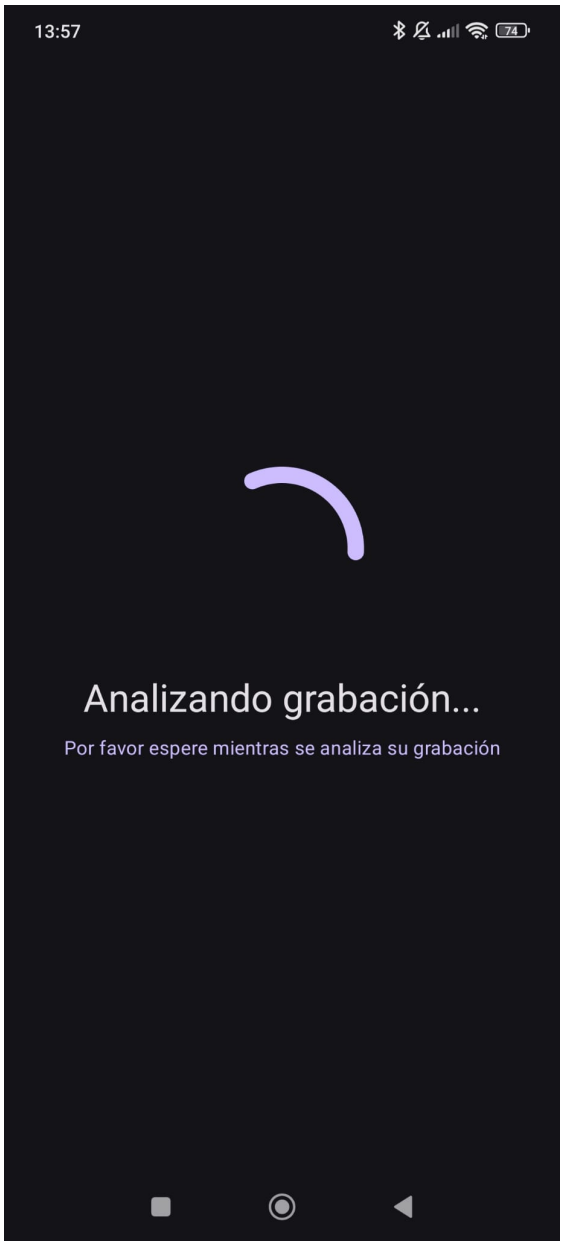


**Figura 5.10:** *Pantalla de carga de la aplicación que aparece al acabar la prueba mientras el modelo de inteligencia artificial analiza el audio de la prueba.*

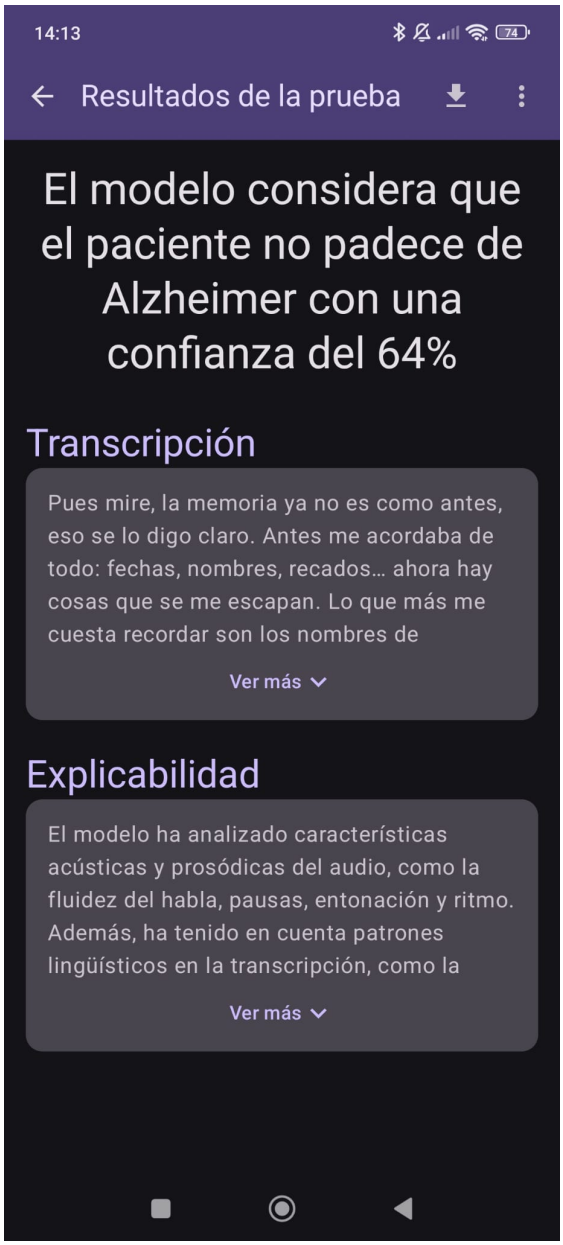


**Figura 5.11:** *Pantalla de resultados en la que se ve el resultado del modelo así como la transcripción de la prueba y la explicabilidad del modelo.*